

Specialized Domains

Chapters 21–23. Analog and mixed-signal verification; safety verification; security verification.

Three markets, three obligations. Each one takes something you have relied on for twenty chapters and removes it.

What you will be able to do

- State an analog check as a *band*, a *window* and a *condition set* — and say what the band is a percentage *of*.
- Read a failure analysis as a verification input, and say why a fault count is not a diagnostic-coverage metric.
- Tell a fault that is *safe* from a fault your workload never excited — and never let the second be recorded as the first.
- Verify a safety mechanism the way teams forget: that it stays *silent* on a healthy part.
- Write a threat-model row, recognise the one you cannot close, and say so rather than widening it.

THE REGISTER SHIFT

Parts I–V asked whether the evidence was enough. Part VI asks a question with a third party in it: *enough for whom?* A continuous quantity, a safety assessor, an adversary. Each arrives with its own idea of what counts as evidence, and none of the three accepts the other two's.

SAY *These are not three techniques. They are three markets — and a market is a set of obligations you did not choose.*

Three chapters, three things the market takes away from you

Twenty chapters of practice, resting on assumptions the tools never made you state

a value is exact · a specification sets the scope · the reasoning lives in the room

21 · The continuous world

there is no bit-exact oracle

22 · A standard is watching

the argument is the deliverable

23 · There is an adversary

no denominator, so no percentage

CH 21

The oracle stops being an equality. *Correct* becomes a **band**, and checker, coverage bin and regression verdict are all rebuilt on it — so a mixed-signal regression's verdict is a property of the design *and* of every tolerance somebody typed into a checker.

CH 22

The reasoning that connects evidence to “good enough to ship” stops living in the heads of the people at the table. You are no longer producing results; you are producing **the premises of somebody else's argument**.

CH 23

The specification stops setting the scope. The property is not “the design does X” but “the design cannot be made to do Y” — where **Y is chosen by someone who read the specification looking for what nobody wrote down**.

SAY Read the third row against the first two. Twenty-two chapters assumed a document defined the job. Chapter 23 is what happens when an opponent defines it instead.

21

Analog and Mixed-Signal Verification

The digital abstraction stops here. The oracle, the checkers, the coverage model and the pass/fail rule that decides whether a regression passed all have to be rebuilt on a foundation where *correct* is a band rather than an equality.

Three engineers spend a week looking for the bug. There is no bug.

```
assert (adc_code == expected_code);
```

It fails on nearly every run — not by much: one code, occasionally two. One least-significant bit against the analog front-end's 1.8 V reference is $1.8 / 4096 \approx 439 \mu\text{V}$, and 439 μV is smaller than the amount by which the model, the schematic and the eventual silicon are *designed* to disagree. The comparison was never going to hold.

WHAT HAPPENS NEXT IS THE REAL DEFECT

Someone changes the check to ± 2 LSB, the regression goes green, and the number is never written down anywhere but that line of code. Six months later a run fails at 3 LSB and nobody can say whether that is a bug, a modelling artifact, or an unlucky seed — because nobody knows where the 2 came from. **A debugging convenience has quietly become the specification.**

SAY The instinct to widen the check is correct. Everything wrong here happens in the ten seconds after it.

The verdict has three parts where a digital verdict has one

PART OF THE VERDICT WHAT IT FIXES, AND WHAT GOES WRONG WITHOUT IT

A band	Absolute (± 10 mV) or relative ($\pm 2\%$) — and the choice is a trap in <i>both</i> directions. Next slide.
A window	Analog outputs settle. A comparison made before the settling time has elapsed is checking a transient; one made at an arbitrary time is checking whatever the model's step size happened to produce. Every analog check needs a stated moment or interval at which it is valid.
A condition set	Supply voltage, temperature, process corner, input frequency, load. A tolerance without the conditions under which it holds is not a specification.

SCENARIO • THE SAME FOUR FIELDS, AS A PLAN ROW

The PLL must be within **100 ppm** of target after lock. The row states the band (100 ppm), the window (1024 output cycles beginning 20 μ s after the lock request), the conditions (nominal supply, three temperature points, three process corners), and the *source* of the band — the tightest frequency accuracy any consumer of the system clock demands. The checker reads all four from a package, not from literals. **Why:** when a run fails at 140 ppm the conversation is about which requirement is violated and whether the band or the design should move. Without the row it is about whether 100 was ever the right number, and it has no ending.

A digital plan row closes when a coverpoint is hit or an assertion is proven. A mixed-signal row closes when a measured quantity sits inside a stated band under stated conditions, **with a named source for the band**. Scattered as literals through the testbench, those numbers leave the team unable to answer the two questions that matter in a bring-up crisis: *where did this tolerance come from*, and *who approved it*.

SAY Those PLL numbers are that chapter's illustration, not a house constant. What transfers is the four fields, and that the fourth one is a name.

A band is only as meaningful as the thing it is a percentage of

```
package afe_spec_pkg;
// Bands for the front-end: one place, one owner,
// quoted everywhere.
localparam real V_REF = 1.8; // reference, volts
// Model-vs-schematic band: 1% of FULL SCALE, named
// in register row MDL-AFE-ADC-01.
localparam real V_CORR_ABS = 0.01 * V_REF;
endpackage

// Correlation runs only: model vs transistor level.
always @(posedge sample_done)
  if (corr_run && !(v_md1 inside {[v_ckt +/- V_CORR_ABS]}))
    $error("AFE model outside its correlation band");
```

The correlation check uses the **absolute** token, and that is the whole argument. A band taken as a percentage of the *measured value* shrinks with the signal and collapses near zero. The same percentage of the *fixed full-scale reference* is a constant band across the whole range.

AND THE TRAP RUNS BOTH WAYS

An absolute tolerance on a quantity that legitimately spans three decades is **meaningless over most of its range**. The reference point is a choice with a failure mode on each side — so state which one you picked, and against what.

Coverage inherits the same problem. A bin naming a single real value can fail to be hit at all, because floating point has limited precision: `0.1 + 0.2` yields `0.30000000000000004`, which is not in a bin covering `{0.3}`. A tolerance token attached to a single value defines an implicit range a real measurement can land in.

SAY None of those numbers is a bare literal. Each is a named constant with a comment, and the correlation band names the model-register row behind it — which is the entire point. `corr_run` keeps the nightly regression and the correlation campaigns apart in one file.

No matrix solve, therefore no convergence failure — that is the whole value proposition

An *analog* model describes the current-versus-voltage relationship between nodes, and the solver finds all node voltages and branch currents by Newton–Raphson iteration with matrix inversion, choosing each timestep from accuracy criteria. A **real** model has no matrix solution at all: the output is computed directly, the model itself decides when to perform each internal computation, and there is no continuous-time operation — only sampled, clocked or event-driven updates, inside the ordinary event-driven simulator.

DESIGN	COMPARED AGAINST	RESULT	YEAR
A major analog block with filtering and fault detection	SPICE netlist	24× faster, 0.3% gain inaccuracy at 100 MHz	2011
PLL loop filter	Verilog-AMS	90× faster (47 s vs 1 h 8 min 32 s)	2015
Delta-sigma ADC	Verilog-AMS	230× faster (0.4 s vs 92.5 s), ENOB 11.515 vs 11.72	2015

DO NOT CHAIN THESE

Three ratios, two independent sources, three different baselines — one against SPICE, two against Verilog-AMS models of the same blocks. They measure separate things. **Any composite figure built by multiplying them is a new claim with nothing behind it.**

THE FIRST ROW IS THE DESIGN RULE, NOT THE HEADLINE

Its 0.3% inaccuracy was *far better than required*, because the tolerances on the checkers were in the range of 5%. The accuracy traded away for the speedup was accuracy the verdict never depended on. **Model to the tolerance the plan row states, and not one digit finer.**

SAY Be blunt about the state of standardisation: there is no standard for how discrete modelling of analog nets should be done, so published examples are one method among several rather than a normative pattern.

Speed is bought with abstraction — know exactly what left the building

Electrical relationships	A real net carries a value, not a Kirchhoff relation. Loading, drive strength, input impedance and current draw are simply absent unless the modeller writes them in. A shared bias line feeding two loads behaves in a real model exactly as it would with one — which is the point at which a whole class of real circuit failures becomes invisible.
Continuous time	The model updates on events; between updates the quantity is a held value, and any transient shorter than the update interval does not exist. Glitches, ringing, overshoot on a supply ramp, and the settling behaviour a tolerance window is supposed to be checking, appear <i>only</i> if the modeller decided in advance to represent them.
Noise, variation, margin	Thermal noise, jitter, offset, mismatch and the spread across process corners are not free consequences of a real model the way they are consequences of a circuit. Each one is a feature somebody wrote, with parameters somebody chose.

NOT AN ARGUMENT AGAINST THE TECHNIQUE

The alternative to abstraction here is not simulating everything — it is simulating nothing. It is an argument for **enumerating which of your checks depend on the discarded physics**, because those checks are checking the model rather than the design.

SCENARIO • THE RADAR RECEIVE CHAIN, AND ITS STATED LIMIT

A real model of the analog path lets the testbench *generate* the chirp: sweep return amplitude across the full input range, inject a controlled noise floor, drive the amplifier into and out of saturation, run it as a constrained-random regression. **The limit:** the model's noise floor is a *parameter*, not a measurement, so a detection threshold tuned against it has been tuned against a cleaner world than the silicon.

SAY *That last sentence is the whole of the next three slides in miniature. The number needs a source, and the source is a correlation run.*

Co-simulation is a campaign, not a regression tier

One simulation, two solvers exchanging values at their shared boundary; where a continuous discipline meets a discrete one, connect modules are inserted automatically. Every time the digital side wants an analog value it must ask the analog solver to interpolate one, and that request is **blocking**: control passes to the analog solver and does not return until the value is available. A sampling clock chosen without thought does not merely add work — it repeatedly stalls the digital engine on the slowest component in the system.

THE RULE THAT SURVIVES CONTACT WITH A SCHEDULE

Co-simulate what real number modelling threw away, and nothing else.

- **Model correlation runs** — the most valuable use, because it is what makes every subsequent real-model run mean something.
- **Failures that live in loading, current and impedance** — precisely the class a real model cannot express and therefore cannot fail on.
- **Power-up, power-down, supply ramps.**
- **One end-to-end signal-path scenario per configuration** — not for bug-finding, but as evidence that the composed system behaves as the real models claim.

SCENARIO • THE SENSOR HUB'S SHARED BIAS REFERENCE

Two channels, one bias reference, firmware may enable them independently. The real model of the bias generator delivers its nominal output regardless of how many channels are enabled, so **a real-model regression will never show a problem**. One co-simulated scenario enables both and checks the reference under the doubled load. The failure is a droop that shifts the first channel's conversion beyond its stated tolerance — a loading failure by construction, invisible to any abstraction that does not carry current. **What it costs:** one scenario, re-run when the bias generator's schematic changes. Not nightly.

Read the previous slide's table backwards for the magnitude: 0.4 s against 92.5 s. At anything like that ratio an hour of real-model regression becomes a multi-day proposition, and a job that had to finish overnight does not finish at all.

SAY The memory PHY's training, owed from Part V: the AMS boundary can discharge the algorithm — convergence against a channel model, termination rather than looping, reachable retry and fallback paths, timing obligations throughout. It cannot discharge the distribution of training outcomes across parts and temperature, which does not exist until there are parts. Two rows; the second closes in the lab.

In analog the schematic is the reference — and nothing mechanically checks the model against it

A model nobody has correlated is a lie you regress against nightly — an expensive one, because it produces green results, at speed, in volume, for months.

Digital

The specification is the reference and the implementation is checked against it. That is what makes a *golden* model golden in this book's vocabulary: it is golden **because it is the specification**. Synthesis then produces the gates, and equivalence checking closes the RTL-to-gates gap mechanically.

Analog

The direction reverses. The schematic is treated as the reference — golden because it is **what gets fabricated**. There is **no analog synthesis**, so the schematic is a creative artifact rather than a derived one, and there is **no equivalence checking** to confirm that a hand-written model matches it.

WHICH RELOCATES THE WHOLE PROBLEM

Digital's risk is a misinterpreted specification, which is why the discipline puts two independent pairs of eyes between specification and RTL. Analog specifications are usually interpreted correctly and are quite simple; the problems are *creating the implementation* and *creating the model*. So analog's second pair of eyes is not an assertion in a testbench — it is **the comparison of two independently created representations of the same block**, the schematic and the model.

SAY Chip-level verification assumes correlated models. Without them, chip-level verification is suspect — and it will not look suspect, because it will be green.

Correlation is a procedure with an artifact — and its result expires

The procedure

- 1. **One self-checking testbench**, written against the block's specification.
- 2. **Run twice** — once against the behavioural model, once against the schematic at transistor level.
- 3. **Compared per port, against a declared tolerance**. In a published programmable-gain-amplifier example: output voltage within **10 mV**, supply current on the supply pin within **1 mA**. The third point is where most teams fall short: the correlation tolerance is part of the specification, **per port, before anyone runs anything**. A match criterion invented after seeing the mismatch is not a criterion.

The model register — one row per model

Correlated against	Schematic rev. 7, transistor level
Criteria	Output code within 1% of full scale (≈ 41 LSB); conversion time within 5%
Conditions covered	Nominal supply, 3 temperature points; corners not correlated
Date and owner	Named analog engineer, with date
If wrong	Every result in the chip-level regression that passes through the converter, and every plan row that closes on one of its measurements
Re-correlate when	The schematic or specification changes; a new operating condition is used; a parameter changes with no schematic change; or a model is edited during debug — the most dangerous, because it happens under pressure and makes a failing test pass

SAY The last three rows are what make the register worth maintaining. A date turns “the model is correlated” into a claim that can visibly go stale, and the consequence field lets a reviewer ask the only question that matters about a model: if this is wrong, what else is wrong with it?

You are almost certainly not verifying the analog block. Name what you *do* own.

The interface contract	Every control line's meaning, every status line's timing, the register map, and the wiring — supply lines, current bias lines, voltage references, and at chip level thousands of digital control lines into the analog domain. A swapped or inverted control line here produces a plausible-looking analog output rather than an obvious failure.
The control and calibration sequences	Trim searches, offset cancellation, lock acquisition, gain ranging. Pure digital logic operating on analog feedback — and the only part of the system whose correctness depends on both a state machine and a tolerance .
The failure handling	What the design does when the analog reports out-of-range, fails to converge, or saturates. A must-not-happen class, systematically under-verified here because the failure conditions are ones the model must be <i>told</i> to produce.
The tolerances themselves	Not the physics behind them — their provenance . Every band in the plan traceable to a specification, and the model register kept current.

SCENARIO • A THIRD CURRENCY, ON THE FLASH CONTROLLER

Read-retry walks a table of reference-voltage offsets. The correctness questions are entirely digital: the table is applied in order, the retry budget is respected, and exhausting the table reports an uncorrectable error rather than silently returning the last attempt. But **the oracle is a probability, not a band**: with correction downstream, a marginal read is not wrong, it is more likely to need correcting. The row closes on a bit-error-rate distribution — a third currency after equality and tolerance, and a reason to **name the currency explicitly in every mixed-signal plan row**.

SAY The sensor hub's wake-up trim search is the second row made concrete: verify the search, not the sensor. Does it terminate for every starting code? What does it do when two codes tie? What happens when the reading never settles — does it converge on a wrong code, or report failure? The trap: a search that always converges in the model may be relying on monotonic behaviour the real sensor does not guarantee — a modelling assumption, and it belongs in the register with the model that carries it.

Two populations, a supply randomised wider than legal, and an owner for each half

Two populations, not one

A nightly regression built entirely on real number models, and correlation campaigns fired by the named triggers. Mixing them produces a nightly job too slow to be nightly, and a correlation result nobody can locate.

Randomise the supply, and cover it

The highest-value analog coverage habit. In a published fractional-N PLL case: nominal **2.5 V**, randomised across **±7%** — deliberately wider than the **±5%** legal band — with three coverage bins, an under-voltage region below 2.375 V, a healthy band from 2.375 V to 2.625 V, and an over-voltage region above. Twenty runs then produce a report that *shows* whether the supply ever left its healthy band. **The numbers belong to that design; the habit is what transfers** — and a team that has never asked the question is usually surprised by the answer.

Split ownership from accounting

The analog engineer writes and correlates the model; the verification team owns its row in the register and its expiry. Neither role works alone: the writer does not see the chip-level runs that depend on it, and the accountant cannot judge whether the physics is right.

THE ECONOMICS, AS A SHAPE RATHER THAN A BUDGET LINE

One consultancy's experience across many mixed-signal chips puts creating correlated analog functional models at **as much as 80%** of the effort of getting analog blocks into chip-level verification, and analog verification as a whole at **typically 20% of the design effort** — rising to 30% or 40% for a team new to it. The same source puts the proportion of analog designers skilled at *reading* models at around **20%**, with fewer able to write them. These are experiential figures rather than survey data and they carry no year: treat them as the shape of the problem.

SAY That skill distribution is why ownership needs an artifact rather than an org chart. The person who can tell whether the model is right is not the person who notices when it is wrong.

22

Safety Verification

She is not asking whether the design works. She is asking whether it *notices when its silicon breaks*, how much of the breaking it notices, and how you know.

Every plan row discharged, coverage closed, regression green — and the first question is about none of it

ELEVEN WEEKS BEFORE TAPE-OUT, AT THE END OF THE TABLE

“Your architecture says the tightly-coupled memory is protected by single-error-correcting, double-error-detecting code, and that a double-bit error is reported to the safety island inside the fault-tolerant time interval. **Show me the faults you injected into that path. Show me the ones the mechanism did not catch. And show me your argument for why the faults you did not inject did not need injecting.**”

Nobody in the room can answer, because nobody was asked to. The error-correcting code was verified the way every other block was — correct data in, correct data out. A perfectly good functional argument, answering a different question.

Process	Activities have prescribed inputs and outputs, reviews prescribed independence, changes prescribed impact analysis. You can no longer quietly delete a test that has become useless, quietly re-scope a coverage model, or quietly upgrade a tool.
Evidence	Green is a <i>state</i> ; evidence is a retained state — tied to a specific design revision and tool configuration, with the raw results kept and not just the summary. From <i>we ran it</i> to <i>we can show, later, what we ran and what came back</i> .
Traceability	Forward from every safety requirement to the thing that discharges it, and — the direction most teams never build — backwards : given a test, name the requirement it serves. A test that traces to nothing is an unexplained artifact in a controlled repository.
And the consequence that outranks all three	The argument itself becomes a deliverable. You are no longer producing results — you are producing the premises of somebody else's argument , and you will be asked whether each premise says what the argument needs it to say.

SAY In 2024, 44% of IC/ASIC projects worked to at least one safety-critical standard or guideline, 63% on FPGA; among those that do, the median spends between a quarter and half of total project time on functional-safety activities. And what teams themselves name as hardest is safety architecture and requirements traceability — ahead of fault list generation and injection. The technique is not the bottleneck; the bookkeeping around it is.

A design can be perfectly correct and still kill somebody

Systematic fault — a design bug

A mistake made during development, present in **every part ever manufactured**, and always permanent. Constrained-random stimulus, formal proof, coverage closure, regression engineering, gate-level simulation, post-silicon bring-up: **almost everything in Chapters 1 to 21 is aimed at this**. A safety standard demands that work too, with more process around it.

Random hardware fault — a physical defect

A latch-up, a marginal via, an aging transistor, a particle strike. Arising during manufacturing or in operation, and either permanent or transient. **You cannot verify it away**. It is a physics problem, and the physics will happen at some rate whatever you do.

WHICH IS WHY THIS WORK FEELS ALIEN

What you *can* do is show that when it happens, the design **notices** and does something safe. For the first time you are not hunting a bug — you are measuring **a mechanism's effectiveness against a population of defects that are nobody's fault**.

SAY Safety verification does not replace functional verification. It adds a second target, and the second target has no bug at the end of it.

You inherit a target — and where no mechanism exists, no campaign can improve its coverage

An **ASIL** — automotive safety integrity level, A through D, with D the most demanding — is **not a property of your block**. It is arrived at by assessing the risk of a potential *hazard*, weighing three judgments: how severe the harm would be, how often the vehicle is exposed to the situation, and how well a driver could control the outcome. A headlamp that fails on a lit motorway and one that fails on an unlit country road are the same electronics and different hazards. The level then travels down — hazard, safety goal, system requirements, requirements on your part, requirements on your block — and by the time it reaches you it is **a number in a requirements document**. You almost never get to argue about it.

SCENARIO • THE REFERENCE SOC HAS NOTHING TO MEASURE

No error-correcting code anywhere. Its 512 KB of tightly-coupled SRAM is fast, four-banked, and completely undefended: a particle strike that flips a bit in a weight buffer produces a wrong answer **with no indication that anything happened**. That was a defensible choice for a low-power edge part whose worst failure is a wrong inference. The answer is not to bolt detection onto it — it is a derivative, the flagship-A, with error-correcting protection on the tightly-coupled memory, the L2 slices and the memory path, a dual-core delayed-lockstep safety island, and monitored reset and clock.

DETECTION IS ARCHITECTURE, NOT A VERIFICATION ADD-ON

There is **no fault campaign you can run on the reference SoC that will improve its diagnostic coverage**, because it has no mechanism whose coverage could be measured. This is the first judgment call of a safety project and it happens *before* verification starts: if the failure analysis says a failure mode needs covering and no mechanism covers it, **the finding is a design finding** — and the later you raise it, the more expensive it is.

SAY One structural fact about the flagship-A, because it recurs: the safety island is built to ASIL D inside a system built to ASIL B. Different parts of one chip carry different targets, and the boundary between them is itself something you verify — a fault in the ASIL B region must not defeat the ASIL D region's mechanisms. That is the reason the safety island exists as an island.

The failure analysis is a verification input, not a document to file

A failure mode, effects and diagnostic analysis works element by element and asks a fixed set of questions of each: in what manner can this element fail; what is the consequence if it does; how likely and how severe is that consequence; and **which safety mechanism handles it**. A typical answer names a mechanism from a small, stable family — error-correcting code on a memory, a dual-core lockstep pair, a software test library, a triply-redundant structure with a majority voter.

IT IS WRITTEN BEFORE YOUR EVIDENCE EXISTS

Produced by a safety engineer, early, from design data and technology data, and it carries an *estimated* diagnostic coverage for each mechanism. **Fault injection is what converts that estimate into a measurement.** If your measurement comes in below the estimate, the architecture's metrics move — and if they move far enough the part misses its target. This is why fault campaigns start late and end badly: they are the last chance to discover that an architectural assumption was optimistic.

ITS GRANULARITY FIXES THE SHAPE OF YOUR FAULT LIST

“The FIFO indicates its flags incorrectly, so data is overwritten or lost, and the mechanism is redundant flag logic” tells you exactly *where to inject* and *which signal is the alarm*. “The block malfunctions” tells you nothing and cannot be measured. **Argue with the granularity while arguing is cheap; you cannot re-cut it in the last four weeks.**

SCENARIO • WALK IT ROW BY ROW, ON A PART YOU CAN ENUMERATE

The brake controller MCU is small enough to be honest about: a lockstep core pair, its protected memories, a watchdog, and a safety island holding the comparator and the alarm aggregation. The verification lead walks the analysis with the safety engineer *before* writing any injection infrastructure, and produces one artifact per row: **where faults are injected, which signal is the observation point, and which signal is the detection point.** On a part this size the walk takes two days and removes almost all the late surprises. On the flagship-A the same walk is a multi-week activity across a dozen sub-teams. **The technique does not change with scale; the coordination cost does, and it dominates.**

SAY That is why large safety programmes staff a safety architect whose full-time job is keeping the analysis and the campaign definition in agreement.

The deliverable is a classification — and ordinary English gets every one of these words wrong

safe	Cannot violate the safety goal — either it cannot affect safety-related logic at all, or its effect is one the system tolerates.
single-point	Violates the safety goal and no safety mechanism is present to catch it. This is the class that must be nearly empty.
residual	Violates the safety goal in a place where a mechanism <i>is</i> present, but this particular fault is outside what the mechanism covers. The honest measure of how good your mechanism is not.
latent	Cannot violate the safety goal on its own, and is neither detected by a mechanism nor perceived. It waits, silently, for a second fault to arrive and combine with it. A dead comparator in a lockstep pair is the canonical example — nothing goes wrong until the day the two cores disagree and nobody is listening.
multi-point	A combination that violates the safety goal, classified by whether the mechanism detects it, whether it is merely perceived, or whether it stays latent.

ASIL	SINGLE-POINT FAULT METRIC	LATENT FAULT METRIC	PROBABILISTIC METRIC
ASIL D	≥ 99%	≥ 90%	< 10 FIT
ASIL C	≥ 97%	≥ 80%	< 100 FIT
ASIL B	≥ 90%	≥ 60%	< 100 FIT

WHICH HALF OF THE FORMULA IS YOURS

The fraction of **safe faults** is established by structural analysis and formal proof. The **diagnostic coverage** terms — the ones that make or break the metric — are measured by fault injection. Every threshold on the left is a number your campaign is an input to, and none of them is a number your campaign produces.

SAY Learn these five precisely. “Not detected” is not one of them, and the next two slides are about why.

Your campaign counts faults. The metrics are ratios of failure *rates*.

Your campaign injects a number of faults and reports how many were detected, and the temptation to divide one by the other and call the quotient your diagnostic coverage is overwhelming. **It is a unit error.**

Each failure mode of each design element carries a **base failure rate** computed from design data and technology data, and the metric formulas combine those rates — the safe fraction, the residual fraction, the multi-point fractions — into a single number for the element and then for the part. Your campaign's detected fraction is **an input applied to a weighted quantity, not the quantity itself**.

THE PRACTICAL CONSEQUENCE

Two campaigns with *identical fault counts* can produce different metrics. A fault list that over-samples a large but low-rate structure while under-sampling a small critical one misleads you in a direction **no coverage report shows**.

SO DO TWO THINGS, FROM THE BEGINNING

Agree the fault list with the safety engineer **per failure mode**, and keep the per-mode results separate all the way to the report. **A single aggregate percentage across a whole block cannot be composed with anything** — which is exactly what makes it useless to the person who has to compute the architectural metric.

SAY Nothing in the toolflow warns you about this. The tool will happily divide two numbers it has, and the quotient will look like a metric.

Good machine, faulty machine — and three words that do not mean here what they meant earlier

Make a second copy of the design with one hypothetical fault inserted — the *faulty machine*, against the unmodified *good machine* — and run the same workload on both. At designated strobe times, compare designated signals: **if a signal differs between the two runs, the fault has reached it**. Two kinds of signal are designated, and confusing them is a common beginner error.



THE WORD HERE	WHAT IT MEANS IN THIS CHAPTER	WHAT IT MEANT EARLIER IN THIS BOOK
fault injection	Corrupting the design's own internal state — a net, a flip-flop — to model a physical defect no interface can produce.	Not <i>error injection</i> , which Chapter 2 introduces and Chapters 3, 8 and 10 practise: driving erroneous stimulus at an interface . A safety assessor means only the first.
redundancy	Duplicated or triplicated silicon — two cores computing the same thing, three copies of a register voted bit by bit.	Not Chapter 2's sense, where redundancy is a second engineer's independent reading of the specification .
lockstep	The hardware pair — two cores running the same instruction stream, watched by a comparator.	Not Chapter 3's sense, where an instruction-set simulator runs in lockstep with the RTL as a reference model .

SAY Same words, different mechanisms — and mixing them produces sentences that sound right and mean nothing. In the published campaign the observation point is the read-data port and the detection points are the single-error, multi-error and any-error signals on the block's boundary. That choice is not incidental bookkeeping; it is the definition of what you are measuring.

A fault that is not detected is not the same as a fault that is not dangerous

Cross the two axes and you get four verdicts. *Observed and detected*: the mechanism caught it — it corrupted something and the alarm went off. *Detected but not observed*: the alarm went off without anything being corrupted — not dangerous, if noisy. *Neither*: it did nothing visible **in this run**. And **observed but not detected** — it corrupted the design's output while the mechanism stayed silent. **That last class is the dangerous one, and it is the whole point of the campaign.**

THE DISTINCTION THAT SEPARATES A COMPETENT SAFETY ENGINEER FROM A DANGEROUS ONE

A fault never observed in your campaign has two possible explanations, and **your campaign cannot tell them apart**: either it is genuinely safe — it cannot affect the safety goal under any stimulus — or it is dangerous and your workload simply failed to excite it. Both look identical in the results database. Both are a row that says “not observed”. Which explanation you choose is the largest single lever on your reported metrics: **classifying every un-excited fault as safe will get you to any number you like**, and an assessor will ask for exactly the argument you did not make.

STRUCTURAL AND FORMAL ANSWER “UNDER ANY STIMULUS”

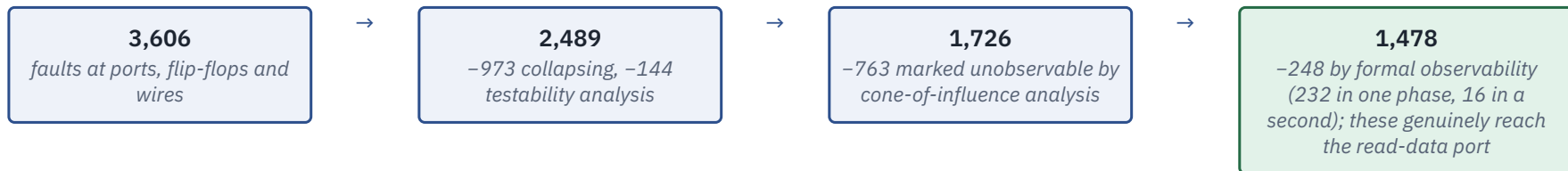
A fault outside the cone of influence of every observation point cannot reach one under any workload whatsoever — a property of the netlist, established statically. A fault a proof engine shows can never cause a difference at the observation point is **untestable by construction**, not merely unstimulated. **Real safe faults. Exclude them without apology.**

SIMULATION ANSWERS “UNDER THIS STIMULUS”

A fault your workload did not excite is a fault about which you have learned **nothing**. It is not evidence of safety; it is **absence of evidence**, and it must be resolved by better stimulus, by proof, or by a recorded engineering judgment with a name attached.

SAY *Keep those two apart every time you prune, and the pruning is defensible. Blur them and the campaign is theatre.*

Nothing in the chain from 3,606 to 1,478 is a guess



A **collapsed** fault is dropped because an equivalent *prime* fault stays in the list to represent it. Every other fault is dropped by an argument that holds **under any stimulus whatsoever**. That is what makes the reduction defensible — and it is why the order matters: **cheap arguments first**, so the expensive engine only ever sees what the cheap ones could not settle.

THE CAMPAIGN'S FINAL CLASSIFICATION, OVER THE 1,726 FAULTS THAT ENTERED DETECTION ANALYSIS

225 neither observed nor detected · **23** detected without being observed · **966** both observed and detected · **512 observed but not detected** — faults that corrupt read data while the error signals stay quiet.

Those 512 are the result. Everything else in the campaign existed to isolate them.

SAY Every net, port and sequential element is a fault site, and each carries at least two permanent fault models plus transient ones — on a large block the raw count runs to millions. This block was small enough to publish, and it was only almost small enough to prove: the memory on its periphery had to be black-boxed and abstracted first, under an abstraction whose error can only make a fault look more observable than it is. An abstraction whose error runs the other way is not an abstraction; it is a bug in your evidence.

A safety mechanism is a design, and it has bugs

It was written by the same people, under the same schedule, from the same specification as the logic it protects. It gets less verification attention than the functional path because **it does nothing in the nominal case**, and it integrates last because it depends on everything else. A safety mechanism is therefore, structurally, **the least-verified block on the part — and the one whose failure the metrics assume away**.

1 · Does it fire when it should?

the fault campaign — the easiest of the three, answered as a by-product



2 · Does it fire *in time*?

the budgeted interval is part of the requirement



3 · Does it stay silent when nothing is wrong?

the question teams forget

LATE IS NOT PARTIAL CREDIT

A mechanism must detect and react **within a budgeted interval measured from the moment the fault occurs**, and must run continually while the device operates. So a campaign classifies detected-in-time against detected-late, and **a late detection is an undetected fault with a misleading log message**. The simulation therefore cannot stop at the first difference — set the post-divergence budget from the timing requirement, not from what makes the run finish quickly.

THE ASYMMETRY THAT EXPLAINS THE SKIP

Proving a mechanism *fires* means injecting a fault — deliberate, scripted, easy to demonstrate. Proving it *stays silent* means running the design's **entire legal operating space** with the alarm never asserting: an obligation with no natural stopping point. A mechanism with false positives is a reliability defect, often worse than the fault it guards against, because it takes a *healthy* system to a safe state — a warning lamp, a limp mode, a dealership visit.

SAY The sensor hub's clock monitor makes the third question concrete. The domain it watches legitimately changes frequency on a low-power transition, legitimately stops during power-down, and legitimately restarts with a ramp — every one of those a healthy event that looks exactly like the failure the monitor exists to catch. The obligation is a cross of every legal power and clock transition against the alarm, with the alarm asserted as an error.

The assertion is worthless without the cover

```
// Bound to the error-correcting wrapper on the
// flagship-A's memory. inject_active is the campaign's
// own control: high only while a fault is deliberately
// present. Every other cycle is a claim about health.
property p_no_spurious_ecc_alarm;
    @(posedge clk) disable iff (!rst_n)
        !inject_active |-> !(ecc_err_single || ecc_err_double);
endproperty
a_no_spurious_ecc_alarm : assert property
    (p_no_spurious_ecc_alarm);

// The mechanism must also be ALIVE, not merely quiet:
// a mechanism that can no longer fire is a latent
// fault, and silence is what it looks like.
c_ecc_double_reported : cover property (
    @(posedge clk) disable iff (!rst_n)
        inject_active && ecc_err_double
);
```

WHY THE ASSERTION ALONE PROVES NOTHING

A wrapper whose error outputs *never* assert — a design bug tied them off, or a fault killed the mechanism — passes

a_no_spurious_ecc_alarm on every run of every test, forever.

Silence is exactly what a dead mechanism looks like, and a dead mechanism is the latent fault of the classification. The cover is what makes the silence mean something.

AND INJECT_ACTIVE IS DOING REAL WORK

Without that qualifier the assertion fires on every campaign run, so the team disables it during campaigns, so it protects nothing during exactly the runs where the protected path is most exercised. Gate it on the injection control and it holds *in campaign runs too* — everywhere the campaign is not currently corrupting something.

SAY The instrument cluster generalises it. Frozen-frame detection watches for content that stops changing — but a correct cluster is legitimately static most of the time, so the mechanism cannot be a frame comparison; it needs an injected, known-varying region the renderer must update every frame. The alarm condition must be a proposition that is false exactly when the design is healthy, never a heuristic that happens to correlate with health. When you find yourself verifying a heuristic, the finding belongs back with the architect — early.

The obvious lockstep test passes on a design with a serious defect

Two cores, one comparator. Inject a fault into one core, check the comparator flags a mismatch. It passes — and it establishes almost nothing, because what makes a lockstep pair effective is not that a mismatch is detectable. It is that **the two cores fail independently**.

WHAT THE MECHANISM ACTUALLY IS

On the flagship-A the pair is **delayed**: the shadow core runs some cycles behind, so a disturbance hitting both at the same instant hits them at *different points in their execution* and produces a mismatch instead of two identical wrong answers. **That delay, and the separation behind it, is the mechanism.**

- Does the **delay pipeline** preserve the comparison across every stall, flush and interrupt?
- Do the two cores share any resource — a clock buffer, a reset synchroniser, a power rail, a memory port — **whose single failure would corrupt both identically**?
- Does the comparator's **own** failure get caught by something else? That is the latent-fault question again, and its usual answer is a periodic self-test that provokes a mismatch on purpose and confirms the alarm.

AND WHERE THE ANSWERS COME FROM

None of that is found by injecting a fault into one core. It is found by asking what a *single* physical event could do to *both* halves at once — the analysis of dependent failures, a discipline of its own that a verification engineer contributes to rather than owns. **What you owe it is the list of shared resources your netlist actually contains, which nobody else can produce.**

SAY Notice the shape repeating from the previous two slides: the interesting obligation is never the one the mechanism was designed to satisfy.

The tool is part of the evidence — and qualification is scoped to a version, a use case and a configuration

Version pinning becomes a safety obligation	The exact tool build is part of the evidence, not part of the environment. Chapter 12's run manifest — seed, design revision, testbench revision, exact build, configuration, host — stops being good practice and becomes the atom of the evidence trail .
A mid-project upgrade is a re-argument, not an IT ticket	When the vendor ships a fix you need, the question is not whether it installs but what evidence produced under the old version must be re-established . Teams that have not thought this through discover it in month nine.

PLAN ROW	OBLIGATION	CLOSURE METRIC
FA-SM04a	Every injected double-bit error raises the alarm to the safety island within the budgeted fault reaction time	Fault classification report complete; dangerous-undetected list itemised and dispositioned
FA-SM04b	The mechanism raises no alarm on healthy traffic, including low-power entry and exit	<code>a_no_spurious_ecc_alarm</code> enabled and passing in 100% of regression runs
FA-SM04c	The classification's residue is argued, not assumed	Zero faults in state <code>not observed, unargued</code> at sign-off

SAY A third consequence, less obvious than the two above: using a qualified tool outside its qualified use case buys nothing — the same simulator driving a flow the qualification never contemplated is a tool you happen to trust. Row `FA-SM04c` is the honest name for the plateau: it does not promise that every fault is safe, it promises that no fault is silently assumed safe. And keep two questions apart, because answering one with the other is a recognisable move. Tool qualification asks whether the software you ran can be trusted. Flow qualification asks whether the environment you built can fail — answered by injecting systematic faults into the design deliberately and measuring what the environment detects, where the undetected ones point at a missing checker rather than a missing test. When a hole survives it goes into a waiver with Chapter 7's five fields; a safety project changes only who signs it. The safety engineer owns the residual risk; the verification lead owns the accuracy of the statement.

23

Security Verification

The functional plan answered its own question completely: the engine does what the specification says it does. The security question is a different one — *can the engine be made to do something the specification never mentioned?*

Signed off twice. The second review opens with a question the plan does not contain.

THE QUESTION

What would an attacker want from this block, what are they able to do, and where do those two meet? — and within an hour the room has produced obligations with no rows behind them.

- Nothing in the plan says what the engine's completion **latency** may and may not depend on.
- Nothing says what becomes of a working key when the key ladder's power domain goes down **in the middle of an operation**.
- Nothing says whether the debug port — once opened by the authenticated unlock path everyone trusts — **can reach** the key ladder, as opposed to merely not being *wired* to it.
- And nothing anywhere in the plan **names an adversary**, which is why none of those questions had a reason to be asked.

HOW WIDE THE GAP CAN BE

A published information-flow study took an encryption block from a public repository — unmodified, fully functional, encrypting correctly across **thousands of test vectors** — and checked one rule against it: *the key must not flow to the data output*. **The rule fails. The key, exclusive-ORed with the data, reaches the output pin as plaintext.** Every functional test that block will ever run continues to pass.

SAY None of that is a failure of the functional plan. The functional plan answered its own question completely. No quantity of the first kind of evidence accumulates into the second.

Four consequences, structural rather than a matter of degree

The properties are negative

“This key never reaches that bus” is a claim about the **complement** of the design's intended behaviour. You cannot establish it by demonstrating intended behaviour, however much of it you demonstrate.

Rarity stops being a defence

Risk-driven verification allocates effort by likelihood times cost, and likelihood is computed against a stimulus distribution the team controls. **An adversary is not a distribution. An adversary is an optimiser with a budget**, who will spend a month arranging exactly the four conditions your generator would only ever have stumbled into — and who, having arranged them once, can arrange them on demand.

The failure is often functionally silent

A confidentiality breach need not corrupt any output. In Chapter 3's vocabulary: the bug is *activated* and it *propagates* — propagation to an observable point is not the detection mechanism here, **it is the failure** — but nothing *checks*, because no checker was ever written for a value that was correct.

Coverage does not close

There is no enumeration of “everything an attacker might try”, so there is no denominator, so there is no percentage.

AND THE OBVIOUS APPROACHES UNDER-DELIVER, EACH IN ITS OWN WAY

Architecture review is manual and rarely reaches the implementation as built. Red-team exercises are essential for the finished product and find nothing before silicon exists. Formal belongs inside a larger strategy and does not scale to a large design running hardware and software together. Directed tests take substantial effort to write and mostly re-confirm vulnerabilities already known. **None is a bad technique — each simply has its strength somewhere other than where an adversary's advantage lies**, so a plan built on any one alone inherits precisely its blind spot.

SAY The second row is the one that should unsettle you. Prioritising by unlikeliness is load-bearing for every functional plan in this book, and it is void inside the threat model's scope.

There is no security-coverage percentage — and the reason is not a tooling shortfall

The output of the process is a **threat model**, a set of properties discharged against it, a set of properties *not* discharged with their reasons, and a residual-risk statement someone signs. A chapter that promised a coverage number would be lying.

WHY THERE IS NO NUMBER

No enumeration of what an attacker might try $\Rightarrow$ **no denominator** $\Rightarrow$ no percentage. Functional metrics — line, toggle, FSM — correlate only loosely with an attacker's search space, and the field has no settled, tool-friendly definition of security coverage. **This is not a temporary tooling shortfall: it is what happens when the space being covered is defined by an opponent rather than by a document.**

WHICH MAKES THE RESIDUAL A DIFFERENT ANIMAL

The shape is Chapter 7's sign-off with one difference that matters: the residual is **not merely unquantified, it is unquantifiable** — because you cannot count what you have not enumerated.

THE ONE THING THAT TRANSFERS — AND IT IS MOST OF THE JOB

Everything downstream of the threat model is verification: properties, assumptions, stimulus, coverage, waivers, sign-off. Once someone has written “*the working key must not be observable at the debug port under adversary capability C3*”, the question of how to establish that is a question this book has spent twenty-two chapters on.

The specialist knowledge is concentrated almost entirely in one artifact. So write that artifact first, and write it as an engineer.

SAY That is uncomfortable, and it is why the discipline leans so hard on structure: if you cannot bound the space, be explicit about which part you looked at, and make each look exhaustive within its scope. Every practice in this chapter follows from that trade.

Assets, capabilities, boundaries — and each is useless without the other two

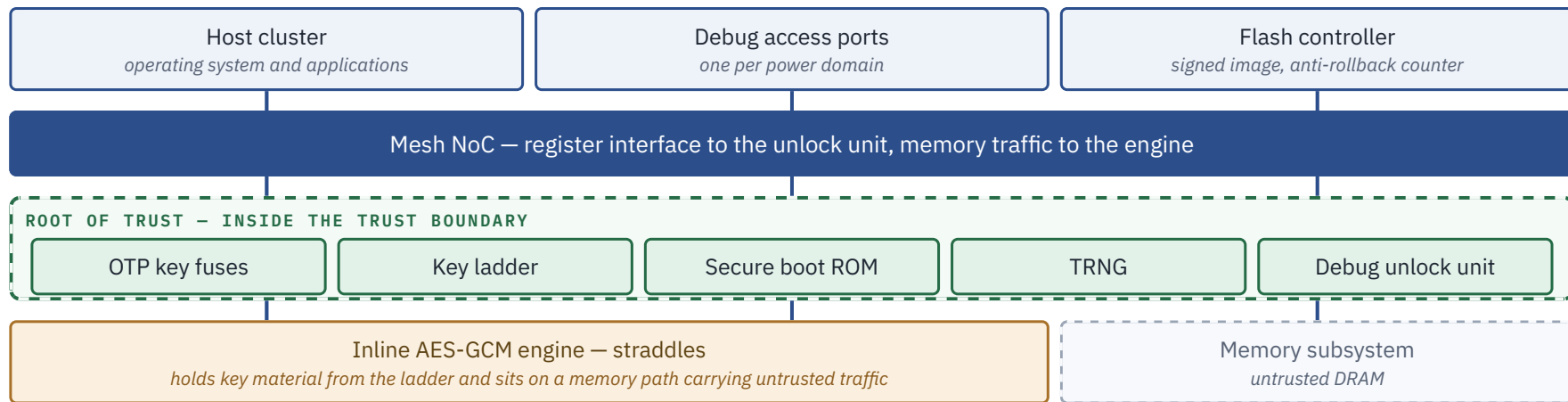
Assets are identified first, because where that is left late the fixes land in the far more expensive post-silicon stage. The obvious ones are cryptographic; **the non-obvious ones matter more**. The firewall and address-decode configuration that enforces every other protection is an asset: if an adversary can rewrite the enforcement, no other lock needs breaking. And an asset need not be secret — the anti-rollback counter is *public by design*, and everything depends on nobody being able to **decrement** it, so what it needs is **integrity**, not confidentiality.

Capabilities: write capabilities, not attacks — attacks are an open set, capabilities are enumerable, **and only an enumerable list can be closed over**. **Boundaries** are the surfaces where the tier changes, and they make the first two lists checkable: a property is always “**asset A does not cross boundary B under capability C**”.

C1	Unprivileged software on the host cluster: loads and stores, programming what it is allowed to program, and <i>timing itself</i> .
C2	Privileged software: everything in C1, plus configuration of the interconnect, the MMU, the power controller and the debug fabric.
C3	Physical access to the package pins: probing debug, measuring supply current and emission, perturbing clock and supply.
C4	Invasive physical access, to decapsulation and direct die probing. Declared out of scope — with the reason written down.

SAY The reason for the C4 exclusion: countering it is a packaging, sensor and countermeasure-design problem whose evidence does not come from RTL verification, and pretending otherwise would put rows in a plan that no pre-silicon activity could ever discharge. An out-of-scope declaration with a reason is engineering. An out-of-scope declaration without one is where the next escape lives.

A figure that boxes a straddling block on one side has already told a lie about it



WHAT THE FIGURE ASSERTS — EXACTLY ONE THING

Which blocks sit inside the trust boundary. Everything drawn outside it is reachable, *in some mode*, by an adversary at C2 or above. The inline engine is deliberately outside the box and is the interesting case, because it is the one component that genuinely straddles.

SAY This is not a diagram detail. The boundary supplies the third field of every row in the plan, so a block placed on the wrong side of it silently rewrites the capability tier of every property that mentions it.

Read the last column first

ID	ASSET • BOUNDARY	CAP.	OBLIGATION	DISCHARGED BY
TM-01	Device root key (fuses) • root-of-trust register interface	C1	No sequence of register accesses makes fuse content observable outside the ladder	Information flow, RTL
TM-02	Working key • memory path through the engine	C2	Key material influences no value on the memory path except through the cipher	Information flow + negative connectivity rows
TM-03	Working key • debug access port into the root-of-trust domain	C3	A debug request addressing the key region is never granted, unlocked or not	Formal, on the always-on control logic
TM-04	Working key • power-domain edge of the root of trust	C2	Powering the domain down destroys the key; leaving the unlocked state requests a wipe	Formal + power-aware simulation
TM-05	Anti-rollback counter • flash controller register interface	C2	No command sequence decreases the counter	Formal, on the counter's control logic
TM-06	Working key • the engine's physical emissions	C3	No key material is recoverable from the engine's power signature	argued, not verified

Three rows name a single technique that produces evidence, two name a pair, and **TM-06 names none**, because at RTL there is no honest way to discharge it. That row is not a defect in the table; *it is the table doing its job*. **A threat model that contains only rows you know how to close is a threat model that was written backwards from the tools you own.**

SAY Treat it exactly as Chapter 5 treats the verification plan: versioned, reviewed, owned, with traceability from each row to its evidence. Design changes invalidate rows constantly and almost never visibly — a performance counter made readable from user mode, a new DMA channel given an address range for convenience. A boundary just moved, and yesterday's report is still green. And a finding at a module boundary is conditional on integration: “this block does not enforce X” and “this SoC is vulnerable to Y” are different claims with different evidence.

Confidentiality quantifies over *pairs* of executions. That is a mismatch of shape.

Both shapes are statements about flow

Integrity is violated when an unauthorised agent — software or physical — can *modify* the state of a target register or memory: untrusted sources must not flow to trusted state. **Confidentiality** is violated when an unauthorised agent can *read* it: secret sources must not flow to observable destinations. Every row of the table is one or the other — which is why sorting assets by which they need was worth the effort.

Now compare a functional property

Chapter 11's 4 KB-boundary property is a claim about **values at a point**: given an accepted incrementing write address, this arithmetic relation holds. It is decidable by looking at *one* execution at that point. A confidentiality property is not decidable that way at all: its natural statement is **change the secret and nothing the adversary can see may change** — and a concurrent assertion evaluates sampled values from one execution.

YOU CANNOT WRITE AROUND THIS WITH A CLEVERER SEQUENCE

It is a mismatch of *shape* rather than a limitation of expressiveness — and it is exactly why information-flow analysis exists as a **separate technique** rather than as a style of assertion.

SAY Hold that thought for two slides. Everything expressible as a single-execution invariant goes to formal; everything that is really a statement about dependence goes somewhere else.

Proved in seconds — and every one of the six lines needs reading precisely

```
// Root-of-trust debug port: obligations TM-03 and TM-04.
// All signals below are in the always-on domain.
a_unlock_needs_token : assert property (@(posedge clk)
    disable iff (!rst_n)
    $rose(dbg_unlock) |-> token_verified);

a_key_region_locked : assert property (@(posedge clk)
    disable iff (!rst_n)
    (dbg_req && dbg_addr_in_key_region) |-> !dbg_gnt);

a_relock_wipes : assert property (@(posedge clk)
    disable iff (!rst_n)
    $fell(dbg_unlock) |=> wipe_req);

// One cover per assertion; the third is the one that matters.
c_key_region_probed : cover property (@(posedge clk)
    disable iff (!rst_n)
    dbg_req && dbg_addr_in_key_region);
```

- `a_unlock_needs_token` constrains **one cycle**. It says nothing about *which* token, and nothing about whether the verification that produced that signal is itself sound. That obligation belongs on the verifier, discharged there rather than enjoyed here.
- `a_key_region_locked` deliberately **does not mention** `dbg_unlock` : guarding a security property with the very state the adversary is working to reach would make it vacuous exactly where it is needed. And note which signal it constrains — the **grant**, not the request. The request belongs to the adversary; **a property written over an adversary-controlled signal is not a property, it is a wish.**

THE THREE COVERS ARE NOT OPTIONAL

`c_key_region_probed` demands something a functional environment will not produce: **a request no cooperative requester would make.** Constraints encode what the interface makes *legal* and what the testbench steers toward, and neither covers traffic whose only purpose is to be refused. An adversary is under no such restraint.

SAY None of the six looks back or forward more than one cycle, and all read always-on signals, so a proof engine settles them in seconds and the result holds for every arrival time of an unlock request. Read `a_relock_wipes` precisely too: it asserts the wipe is requested, not that it completes, which is all TM-04 asks of it here — the row's other half is the power-aware test, which is why the row names two techniques. And reusing the functional environment unchanged is not a saving; it is a silent deletion of the entire threat model, and the most common way a security campaign runs green for a month while checking nothing.

Four parts to a rule — and the exception clause carries the whole argument

Labels are attached to data — secret or public, trusted or untrusted — and their propagation is followed through the design. The label is usually called *taint*, and the mental model is worth stating exactly: **you are not tracking the value, you are tracking dependence**. Anything whose value could differ *because the secret differs* is tainted — whether it holds a copy of the secret, a function of it, or merely a bit that was set because of it.

PART	WHAT IT NAMES	ON THE CRYPTO ENGINE
Source set	the asset, as the signals that hold it	every bit of the key ladder's working-key output
Sink set	what the adversary can observe	bus-readable registers, the debug data register, the trace fabric's input
Activation	when tracking starts	from the cycle a key is loaded, rather than from reset
Exception	the flow that is permitted	the cipher core's ciphertext output

READ THE FOURTH ROW AGAIN — IT IS WORSE THAN AN OVER-CONSTRAINING ASSUMPTION, IN ONE SPECIFIC WAY

In words the rule is: *the working key must not flow to any output, ignoring the cipher's ciphertext output*. Without the exception the rule fails immediately and *correctly* — the ciphertext depends on the key; that is what a cipher is. With it, the rule becomes checkable and **exactly as strong as the exception is narrow**. And an over-constraining assumption looks like a modelling shortcut, so a reviewer who reads assumption lists will challenge it — whereas **an exception clause looks like domain knowledge**, so it reads as competence rather than as a concession and gets waved through. **Widen it by one signal and you have declared a whole port secure.**

SAY “The ciphertext output is declassified because the cipher is trusted” is an argument. “We added *busy* to the ignore list because the rule kept failing” is a defect with a comment on it. Every exception is a row, with a written argument, an owner and an expiry — reviewed exactly like a waiver.

A no-connection result proves non-adjacency. The key's own path is supposed to be there.

What connectivity established

Chapter 15 proved the negative rows for this exact key ladder — no structural path to a bus-readable address, to the debug data register, or to a scan-observable point. That is a **structural** result: *no path exists*.

What it cannot establish

The key bus from the ladder to the cipher core is a path that exists **by design**. Nobody is going to prove it away. The question is whether anything reachable downstream depends on the key *other than through the cipher's intended transformation* — and **no cone-of-influence check can answer it, because the cone is supposed to be there**.

WHICH IS PRECISELY THE FAILURE IN THE OPENING RESULT

Structurally, key and datapath are adjacent — they must be. Functionally, a value carrying the key survived to the output pin. **A team holding a green connectivity report and calling it non-leakage has proved non-adjacency and filed it under the wrong heading.**

SCENARIO • THE GPU, WHERE THE FAILURE IS NEVER A WIRE

Four shader clusters share a unified memory and contexts belonging to different applications time-slice it. Asset: one context's data. Sink: anything the next context can read. Boundary: the context switch. The failure is **residue** — state a functional test cannot see because the next context overwrites it before reading, and that a taint rule sees immediately *because it tracks dependence rather than values*. And note what makes the rule statable at all: **the activation condition**. Without it the rule is “context A's data never reaches memory”, which is false by construction, since writing to memory is the point.

SAY The interesting covers are on the sources, not the sinks: did this run ever put a secret in the source register, ever switch context with live data resident, ever load a key at all. A flow rule evaluated over a run in which no source was ever tainted is a vacuous pass wearing a different hat — green report, nothing tracked. And scope one rule per threat-model row, so every alert arrives already attached to an asset, a capability and a boundary: a screen returning four hundred benign alerts will be ignored within two weeks, and the instinct to loosen the policy is the move that starts deleting real findings.

Sort the channels by whether the leaking quantity exists in your model

An RTL model represents logic values at clock ticks and nothing else. It does not model supply current badly, or approximately. **It does not model supply current at all.**

inside the model

Timing

the good news

A cycle count is a signal-level fact: if the number of cycles between an operation's start and its completion depends on key material, that dependence is *present in the RTL*. Where the specification declares a **fixed** latency, the claim is single-execution and mechanical: `$rose(op_start) |-> ##LAT op_done`, with a cover on `op_start`. Where the latency legitimately **varies**, that assertion is false and the claim becomes a **flow rule** instead: no key bit may flow to the completion signal. **op_done is just another sink**. A timing channel is not a new category of problem; it is a confidentiality property whose sink happens to be a control signal rather than a data bus.

outside the model

Power and emission

the bad news

Nothing in the RTL holds them. Pre-silicon work can only reason over **proxies** — switching activity, performance counters, timing distributions. **Be precise about what a proxy licenses:** switching activity *correlates with* dynamic power; it is not power, and a countermeasure that improves the proxy has been shown to improve *the proxy*. Ranking structures by risk and comparing a countermeasure against its absence are real value. **A leakage verdict is not available at this abstraction, and a report that implies one is worse than none, because it retires a question that is not closed.**

SAY And the capability tier is itself a design decision. Physical side channels are nominally C3 — they need the package. But software-controllable voltage and frequency scaling, and power monitoring, are ordinary features of modern chipsets, and they let an attacker mount fault-injection and side-channel attacks without physical access. The flagship has three operating points, and something selects among them: if that selector is reachable from C1 or C2, a whole row's capability tier has quietly changed, and every property justified by “the adversary would need physical access” is now unsupported. That is the question a verification engineer is placed to ask — not how a power analysis works, but which side of a boundary that selector's enable sits on.

The test is short enough to apply in a review: *can you name the signals?*

A verifiable security property can be written down **entirely as signals in a model you own** — a source set, a sink set, an activation condition and an exception if it is a flow rule; an antecedent and a consequent if it is an invariant.

TM-03	passes as an invariant: <code>dbg_req</code> , <code>dbg_addr_in_key_region</code> , <code>dbg_gnt</code>
the timing claim	passes as a flow rule: <code>op_start</code> and <code>op_done</code> are signals, and “no key bit reaches the second” is an ordinary rule
TM-06	fails — its subject is a physical quantity with no representation in the model . No signal list exists, and nothing will ever settle it. Between those poles sit the properties whose signals exist but whose exception is doing suspicious work , and that is where a reviewer should spend the meeting.

WHAT TM-06 GETS INSTEAD OF A PROOF

A named countermeasure recorded in the design review; a proxy comparison showing it moves the proxy in the expected direction, with its own limitations written down; and a post-silicon leakage assessment on the plan **with a date and an owner**. Chapter 7's conditional sign-off, applied honestly. **What it is not is a green tick** — and that distinction is the single most useful thing to hold clearly in a security review.

SAY Which is also the answer to what someone without a security background contributes. Not attacks, and not cryptography: three questions asked persistently — who can write this register?, what happens to this state across this transition?, what exactly does this exception permit? None requires knowing how a power analysis works. And a security campaign is a campaign in Chapter 12's exact sense: it must compete for emulator slots explicitly, with a named block list and a named window, because an activity described as ongoing background work will lose every scheduling argument it is never present for.

Three domains, three incompatible answers to one question — *what counts as evidence?*

	21 • THE CONTINUOUS WORLD	22 • A STANDARD IS WATCHING	23 • THERE IS AN ADVERSARY
The verdict is	a band — with a window and a condition set, and a named source for the band	a metric against a threshold — $SPFM \geq 99\%$, $LFM \geq 90\%$ at ASIL D, tabulated in advance	a named claim — discharged against a named adversary, or explicitly not
Its unit is	the measured quantity's own units, against a stated reference point	a ratio of failure rates — never of fault counts	none. There is no denominator, because there is no enumeration
The residue is	sourced — a tolerance with a specification behind it and a model register whose <i>date</i> makes it go visibly stale	argued, or fixed — row FA-SM04c : zero faults in state not observed, unargued at sign-off	signed — a residual that is not merely unquantified but unquantifiable

THE SHARPEST CONTRAST IN PART VI IS BETWEEN THE MIDDLE COLUMN AND THE RIGHT ONE

Chapter 22 is **entirely** about percentages against thresholds — tabulated, per level, agreed before the campaign runs. Chapter 23 says there is no percentage, there will not be one soon, and **that is not a tooling shortfall**: it is what happens when the space being covered is defined by an opponent rather than by a document. Carry the two habits into the same review and one of them will corrupt the other — either a security report grows a number nobody can defend, or a safety residue gets waved through as unquantifiable when it is nothing of the kind.

SAY No single chapter states this, and it is the real lesson of Part VI: before you can say whether the evidence is enough, you have to say what kind of thing the evidence is. Get that wrong and every downstream argument inherits the error. Note too that all three end at a name, but at different names — the analog engineer correlates the model while the verification team owns its register row and its expiry; the safety engineer owns the residual risk while the verification lead owns the accuracy of the statement; and the security residual takes one signature on a quantity nobody can count.

The through-line: a market you did not choose decides what counts as done

Chapter 21's market is physics — the digital abstraction stops and the oracle is rebuilt as a band. Chapter 22's is a standard and its assessor — the argument stops living in the room and becomes the deliverable. Chapter 23's is an opponent — the scope stops coming from the specification. None of the three is optional once you are inside it, and none of them accepts the others' evidence.

- 21 Mixed-signal — the band, its reference point, and the model register behind both
- 22 Safety — classify, weigh by failure rate, and argue the residue
- 23 Security — named claims, named gaps, and no percentage

WHAT PART VI DELIBERATELY LEFT OPEN

The analog block itself — its performance, noise, linearity and corner behaviour belong to the analog team, who have tools and instincts you do not. The standards' own texts, which should be read rather than paraphrased. Cryptanalysis, and how a power analysis actually works. And the manufacturing-test flow the fault models were borrowed from.

EIGHT SENTENCES WORTH LEAVING WITH

- A debugging convenience becomes the specification the moment nobody writes down where the number came from.
- A band is only as meaningful as the thing it is a percentage of.
- An analog model is an assumption with a waveform.
- Faults are counted; metrics are weighted by failure rates.
- “Not detected” and “not dangerous” are different verdicts, and only one of them is yours to declare.
- A safety mechanism is the least-verified block on the part and the one the metrics assume works.
- A no-connection result proves non-adjacency; only a flow argument speaks to the paths that exist by design.
- A threat model in which every row closes was written backwards from the tools you own.